

Clasificación de Sonidos de Animales (Gatos vs Perros) mediante Redes Neuronales Convolucionales y Recurrentes Bidireccionales: Comparación de Representaciones en Frecuencia y Tiempo

Samuel Acosta, Santiago Renteria

Programa de Ingeniería Electrónica

Universidad del Magdalena — Santa Marta, Colombia

sdacostap@unimagdalena.edu.co | srenteria@unimagdalena.edu.co

Resumen — En este proyecto se implementa y se comparan dos enfoques para la clasificación automática de sonidos de animales, específicamente de gatos y perros tomados del dataset público Cats vs Dogs Audio Classification, obtenido de Kaggle, para nuestro primer enfoque se emplea una Red Neuronal Convolucional (CNN) que opera con mel-espectrogramas generados mediante la Transformada de Fourier de Tiempo Corto. Para nuestro segundo enfoque se utiliza una Red neuronal Recurrente, que procesa la señal temporal normalizada dividida en frames de 512 muestras con solapamiento del 50%. Los dos modelos se entrenaron bajo las mismas condiciones de partición de datos (70%/20%/10%), optimizador Adam y función de pérdida binary crossentropy, los resultados obtenidos de entrenar estos modelos demuestran diferencias significativas en accuracy, complejidad paramétrica y velocidad de convergencia entre los dos enfoques usados en nuestro proyecto. Adicionalmente, se implementó una interfaz gráfica interactiva en Google Colab para que el usuario escoja que audio quiere probar.

Palabras clave — CNN, BiLSTM, framing temporal, mel-espectrograma, clasificación de audio, librosa, TensorFlow, STFT, aprendizaje profundo, procesamiento de señales.

Abstract — This project implements and compares two approaches for the automatic classification of animal sounds, specifically cat and dog sounds, taken from the public dataset Cats vs Dogs Audio Classification, obtained from Kaggle. Our first approach uses a Convolutional Neural Network (CNN) that operates on mel-spectrograms generated using the Short-Time Fourier Transform. Our second approach uses a Recurrent Neural Network, which processes the normalized temporal signal divided into frames of 512 samples with 50% overlap. Both models were trained under the same data partitioning conditions (70%/20%/10%), using the Adam optimizer and a binary crossentropy loss function. The results obtained from training these models demonstrate significant differences in accuracy, parametric complexity, and convergence speed between the two approaches used in our project. Additionally, an interactive graphical interface was implemented in Google Colab to allow the user to select which audio they want to test.

Keywords — CNN, BiLSTM, temporal framing, mel-spectrogram, audio classification, librosa, TensorFlow, STFT, deep learning, signal processing.

I. DESCRIPCIÓN DEL DATASET

El dataset seleccionado para este proyecto es Cats vs Dogs Audio Classification, disponible públicamente en Kaggle bajo el usuario stealthtechnologies. Se trata de un conjunto de datos de clasificación binaria de sonidos de animales domésticos, diseñado específicamente para tareas de aprendizaje profundo sobre señales de audio.

Tabla 1. Características del dataset Cats vs Dogs Audio Classification.

Parámetro	Descripción
Nombre	Cats vs Dogs Audio Classification
Fuente	https://www.kaggle.com/datasets/stealthtechnologies/cats-vs-dogs-audio-classification
N.º de clases	2 (Cats, Dogs)
Formato de archivos	WAV
Duración por clip	Variable — recortada/rellenada a 3 s fijos
Frecuencia de muestreo	22 050 Hz (remuestreada con librosa)
Estructura	Carpetas /Cats y /Dogs dentro de /DvC
Partición utilizada	70% train / 20% validación / 10% test (stratify)
Etiquetas	0 = Gato, 1 = Perro

El dataset no posee folds oficiales, por lo que se realizó una división estratificada utilizando sklearn.model_selection.train_test_split con random_state=42 para garantizar reproducibilidad y balance de clases en cada partición.

II. MARCO TEÓRICO

A. Señales de Audio y Muestreo

Una señal de audio digital es una secuencia discreta $x[n]$ obtenida por muestreo uniforme de la señal analógica continua a una frecuencia de muestreo F_s . El teorema de Nyquist-Shannon establece la condición mínima para evitar aliasing:

$$F_s \geq 2 \cdot f_{max} \quad (1.0)$$

Para señales de audio con contenido hasta 11 025 Hz, la frecuencia de muestreo de 22 050 Hz utilizada en este proyecto satisface la condición de Nyquist. Cada audio se normaliza a 3 segundos fijos (66 150 muestras), aplicando zero-padding cuando el clip es más corto.

B. Transformada de Fourier de Tiempo Corto (STFT) y Mel-Espectrograma

La STFT calcula el espectro local de una señal aplicando la DFT sobre ventanas superpuestas de longitud finita:

$$STFT\{x[n]\}(m, \omega) = \sum x[n] \cdot w[n-m] \cdot e^{-j\omega n} \quad (1.1)$$

El mel-espectrograma aplica adicionalmente un banco de M filtros triangulares en escala mel, que aproxima la respuesta perceptiva del oído humano comprimiendo las altas frecuencias. La escala mel se define como:

$$mel(f) = 2595 \cdot \log_{10}(1 + f/700) \quad (1.2)$$

En este proyecto se generan mel-espectrogramas de 128 bandas mel con $n_fft=2048$ y hop_length por defecto de librosa, resultando en imágenes de 128×130 que sirven como entrada a la CNN.

C. Framing y Segmentación Temporal

Para el enfoque RNN, la señal temporal normalizada se segmenta en frames mediante una ventana deslizante de longitud $L=512$ muestras y salto $H=256$ muestras (solapamiento del 50%). Cada frame representa la amplitud cruda de la señal en un intervalo de $512/22050 \approx 23$ ms. La normalización z-score se aplica antes del framing:

$$c[k] = \sum \log(S[m]) \cdot \cos(\pi k(m - 0.5)/M), \quad m = 1 \dots M \quad (1.3)$$

El resultado es una secuencia de 257 frames de 512 valores cada uno, con forma (257, 512), que alimenta el modelo BiLSTM como serie temporal pura sin información espectral.

D. Red Neuronal Convolutiva (CNN)

Las CNN extraen jerarquías de características espaciales mediante capas de convolución 2D. Cada capa aplica F filtros aprendibles de tamaño $k \times k$ sobre el mapa de entrada, generando mapas de activación:

$$(x * k)[i, j] = \sum \sum x[i + m, j + n] \cdot k[m, n] + b \quad (1.4)$$

La activación ReLU introduce no-linealidad: $f(z)=\max(0, z)$. El MaxPooling reduce la dimensión espacial conservando el valor máximo en cada ventana 2×2 . El Dropout aleatorio desactiva neuronas durante el entrenamiento para prevenir sobreajuste.

E. Red Recurrente Bidireccional (BiLSTM)

Las LSTM (Long Short-Term Memory) son celdas recurrentes con compuertas que controlan el flujo de información a lo largo de la secuencia temporal. Las compuertas de olvido, entrada y salida se definen como:

$$f_t = \sigma(W_f \cdot [h_t - 1, x_t] + b_f) \quad (1.5)$$

$$i_t = \sigma(W_i \cdot [h_t - 1, x_t] + b_i) \quad (1.6)$$

$$o_t = \sigma(W_o \cdot [h_t - 1, x_t] + b_o) \quad (1.7)$$

Una capa Bidireccional (BiLSTM) procesa la secuencia en ambas direcciones (hacia adelante y hacia atrás) y concatena los estados ocultos resultantes, capturando dependencias contextuales en toda la secuencia temporal de amplitudes.

III. PREPROCESAMIENTO Y ANÁLISIS ESPECTRAL

A. Carga y Estandarización de Audios

Cada archivo de audio se cargó con `librosa.load()` con una frecuencia de 22 050 Hz. Los clips se recortaron o rellenaron con ceros hasta una duración fija de 3 segundos para garantizar un entrenamiento preciso y uniforme de los modelos, la carga y normalización se realizaron de la siguiente manera:

```
SR = 22050; DURATION = 3; N_MELS = 128

y, sr = librosa.load(ruta, sr=SR,
duration=DURATION)

# Zero-padding si el clip es más corto
if len(y) < SR * DURATION:
    y = np.pad(y, (0, SR*DURATION - len(y)))
```

B. Extracción de Features — CNN: Mel-Espectrograma

Para el enfoque CNN, cada señal de audio se transforma en un mel-espectrograma de 128 bandas, convertido a escala logarítmica en decibelios (dB) para simular la percepción auditiva no lineal del ser humano. La imagen resultante tiene forma (128 mels, 130 frames), se hace con este fragmento de código:

```
# Para CNN → Mel-Spectrogram
mel = librosa.feature.melspectrogram(
    y=y, sr=SR, n_mels=N_MELS)
mel_db = librosa.power_to_db(mel, ref=np.max)
# Forma resultante: (128, 130)
X_cnn.append(mel_db)
```

C. Extracción de Features — RNN: Framing Temporal

Para el enfoque RNN, la señal temporal normalizada se divide en frames de 512 muestras con hop de 256 (50% solapamiento). La matriz resultante, procedemos a transponerla para de esta manera obtener la forma de secuencia temporal que se requiere para entrenar el modelo recurrente:

```
# Para RNN → Framing señal temporal pura
y_norm = (y - y.mean()) / (y.std() + 1e-8)
frames = librosa.util.frame(y_norm,
    frame_length=512, hop_length=256).T
X_rnn.append(frames) # shape: (257, 512)
```

E. Análisis Visual de Señales

Es pertinente, en primera instancia realizar un análisis mediante graficas de las señales que vamos a trabajar, la Figura 1 muestra la señal en el dominio del tiempo y el mel-espectrograma para un ejemplo de cada clase de audio tomado, observamos en el espectrograma que los maullidos

de gato presentan patrones de energía concentrados en bandas frecuenciales más estrechas y periódicas, y que los ladridos de perro poseen transientes más amplios y distribuidos en un rango de mayor espectro.

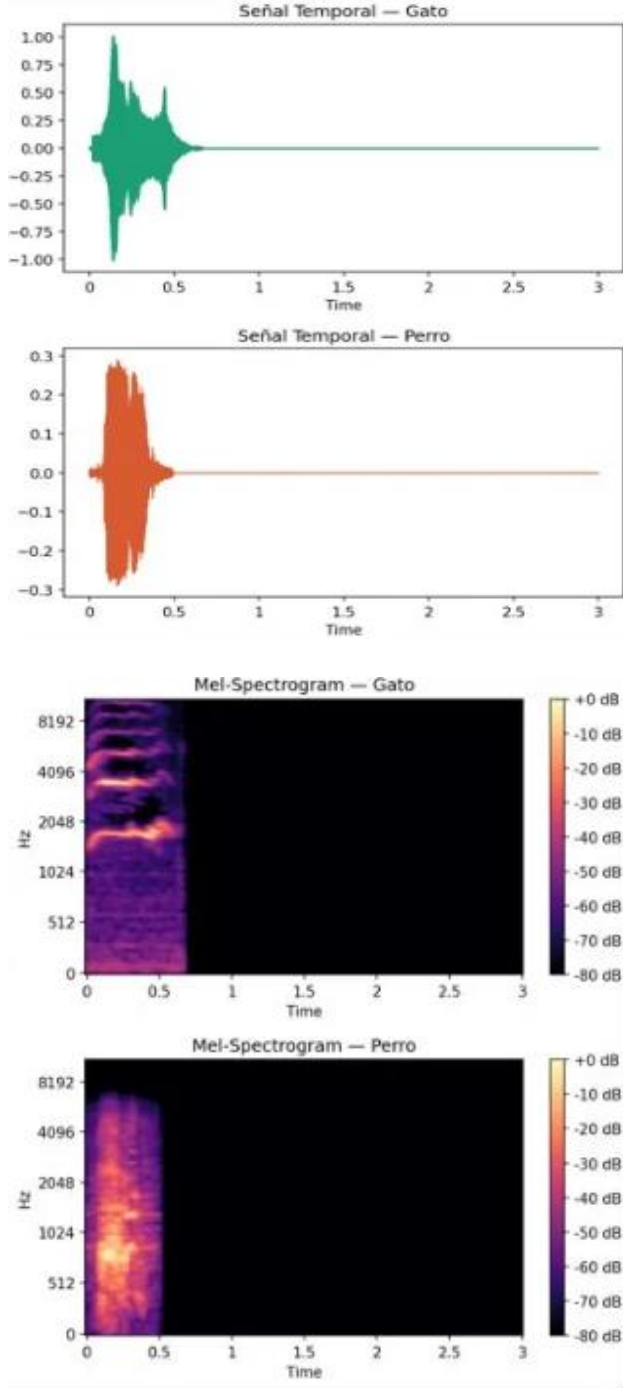


Fig. 1. Señal temporal (arriba.) y mel-espectrograma (abajo.)

Esta diferencia en los espectros de cada tipo de señal, permite justificar el uso de mel-espectrogramas como representación visual para la red CNN, porque los patrones de tiempo y frecuencia de ambas clases se pueden distinguir visualmente, el framing temporal captura la dinámica de amplitud cruda, que nos es útil para

representar la variación temporal del timbre vocal de cada animal como una secuencia.

F. Partición de Datos

se aplicó una división estratificada en tres conjuntos, para el entrenamiento de los modelos. La Tabla II resume la distribución resultante de dicho proceso:

Tabla II. Distribución de los conjuntos de datos.

Conjunto	Porcentaje	Gatos	Perros	Total
Entrenamiento	70 %	368	367	735
Validación	20 %	106	105	211
Test	10 %	53	51	104

IV. DISEÑO DE MODELOS

A. Modelo CNN — Clasificación sobre Mel-Espectrogramas

La red CNN recibe imágenes de mel-espectrograma de forma (128, 130, 1) y aplica dos bloques convolucionales seguidos de capas densas. La arquitectura completa se detalla en la Tabla III:

Tabla III. Arquitectura del modelo CNN.

Capa	Tipo	Filtros / Unidades	Kernel / Pool	Activación
1	Conv2D + padding='same'	32	3×3	ReLU
2	MaxPooling2D	—	2×2	—
3	Dropout (0.25)	—	—	—
4	Conv2D + padding='same'	64	3×3	ReLU
5	MaxPooling2D	—	2×2	—
6	Dropout (0.25)	—	—	—
7	Flatten	—	—	—
8	Dense	64	—	ReLU
9	Dropout (0.40)	—	—	—
10	Dense (salida)	1	—	Sigmoid

```

modelo_cnn = models.Sequential([
    layers.Conv2D(32, (3,3),
        activation='relu',
        padding='same',
        input_shape=(128, 130, 1)),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),
    layers.Conv2D(64, (3,3),
        activation='relu',
        padding='same'),
    layers.MaxPooling2D((2,2)),
    layers.Dropout(0.25),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(1, activation='sigmoid')
])

```

```

1)
modelo_cnn.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy'])

```

La capa de salida del modelo utiliza una función de activación sigmoid y pérdida binary crossentropy, se usaron estas específicamente, porque son apropiadas para la clasificación binaria usada en este caso. El modelo tiene aproximadamente 4.2 millones de parámetros entrenables, dominados por la capa Flatten→Dense.

B. Modelo RNN — Clasificación sobre Señal Temporal Pura

El modelo de red neuronal recurrente recibe secuencias temporales de forma (257 frames, 512 muestras) y se aplican dos capas BiLSTM apiladas, dado que la bidireccionalidad nos permite capturar datos de la señal tanto hacia adelante como hacia atrás en la secuencia temporal.

Tabla IV. Arquitectura del modelo BiLSTM.

Ca pa	Tipo	Unida des por direcc ión	Salida	Activa ción
1	Bidirectional(LSTM)	64	return_sequences=True	tanh
2	Dropout (0.30)	—	—	—
3	Bidirectional(LSTM)	32	return_sequences=False	tanh
4	Dropout (0.30)	—	—	—
5	Dense	64	—	ReLU
6	Dropout (0.40)	—	—	—
7	Dense (salida)	1	—	Sigmoid

```

modelo_rnn = models.Sequential([
    Bidirectional(LSTM(64,
        return_sequences=True),
        input_shape=(130, 40)),
    Dropout(0.3),
    Bidirectional(LSTM(32)),
    Dropout(0.3),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.4),
    layers.Dense(1, activation='sigmoid')
])

modelo_rnn.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy'])

```

El modelo de la red RNN cuenta con aproximadamente 99 000 parámetros, significativamente más compacto en

cuanto a tamaño que la red CNN (4.2 M). esta diferencia en los parametros refleja que la CNN almacena los datos de forma visual del espectrograma completo a través de los filtros, mientras que en la RNN, donde se usó BiLSTM opera sobre la señal temporal cruda dividida en frames de amplitud.

C. Hiperparámetros de Entrenamiento

Tabla V. Hiperparámetros de entrenamiento de ambos modelos.

Hiperparámetro	CNN	BiLSTM (RNN)
Épocas máximas	50	50
Épocas efectivas	13 (early stopping)	20 (early stopping)
Tamaño de lote	32	32
Tasa de aprendizaje	0.001 (Adam)	0.001 (Adam)
Función de pérdida	Binary Crossentropy	Binary Crossentropy
Early stopping	Patience=8 (val_accuracy)	Patience=8 (val_accuracy)
Parámetros totales	~4.2 M	~99 K
Entrada	Mel-Spec (128×130×1)	Frames Temporales (257×512)
Dominio	Frecuencia (imagen)	Tiempo (secuencia)

V. CÓDIGO

A. Preprocesamiento Completo

```

base_path = '/content/AudioDataset/DvC'
clases = {'Cats': 0, 'Dogs': 1}
X_cnn=[]; X_rnn=[]; y_labels=[]

for clase, etiqueta in clases.items():
    for archivo in os.listdir(f'{base_path}/{clase}'):
        y, sr = librosa.load(ruta, sr=SR, duration=3)
        if len(y) < SR*3:
            y = np.pad(y, (0, SR*3-len(y)))
            mel = librosa.feature.melspectrogram(
                y=y, sr=SR, n_mels=128)
            X_cnn.append(librosa.power_to_db(mel))
            y_norm = (y - y.mean()) / (y.std() + 1e-8)
            frames = librosa.util.frame(y_norm,
                frame_length=512,
                hop_length=256).T
            X_rnn.append(frames) # (257, 512)
            y_labels.append(etiqueta)

```

B. Normalización CNN y División de Datos

```

# Normalización individual por muestra
def normalizar(X):
    X_norm = np.zeros_like(X)
    for i in range(len(X)):
        mn,mx = X[i].min(), X[i].max()

```

```

        if mx-mn > 0:
            X_norm[i]=(X[i]-mn)/(mx-mn)
        return X_norm

# División 70/20/10 estratificada
X_cnn_tr, X_cnn_tmp, X_rnn_tr, X_rnn_tmp,
y_tr, y_tmp = train_test_split(
    X_cnn, X_rnn, y_labels,
    test_size=0.30, random_state=42,
    stratify=y_labels)

X_cnn_val, X_cnn_te, X_rnn_val, X_rnn_te,
y_val, y_te = train_test_split(
    X_cnn_tmp, X_rnn_tmp, y_tmp,
    test_size=0.33, random_state=42,
    stratify=y_tmp)

```

C. Evaluación y Métricas

```

from sklearn.metrics import (
    classification_report, confusion_matrix)
import seaborn as sns

# CNN
loss_cnn, acc_cnn = modelo_cnn.evaluate(
    X_cnn_test_r, y_test, verbose=0)
y_pred_cnn = (modelo_cnn.predict(
    X_cnn_test_r) > 0.5).astype(int)
print(classification_report(
    y_test, y_pred_cnn,
    target_names=['Gato', 'Perro']))

# RNN
loss_rnn, acc_rnn = modelo_rnn.evaluate(
    X_rnn_test_n, y_test, verbose=0)
y_pred_rnn = (modelo_rnn.predict(
    X_rnn_test_n) > 0.5).astype(int)

```

VI. RESULTADOS Y ANÁLISIS

A. Curvas de Entrenamiento

En las Figuras 2 y 3 se observan las curvas de entrenamiento, de accuracy y loss de entrenamiento y validación para la CNN y la RNN respectivamente, cuando se realizó el entrenamiento. La CNN convergió en 25 épocas (early stopping activado), mientras que la RNN requirió 12 épocas para alcanzar su mejor val_accuracy, posible en ese momento.

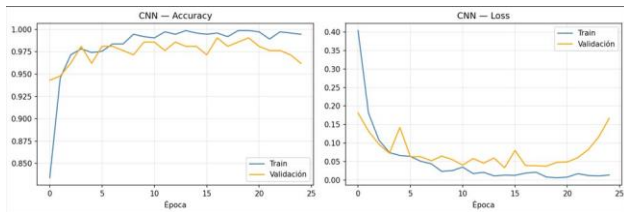


Fig. 2. Curvas de accuracy y loss del modelo CNN (entrenamiento y validación). Convergencia en época 25.

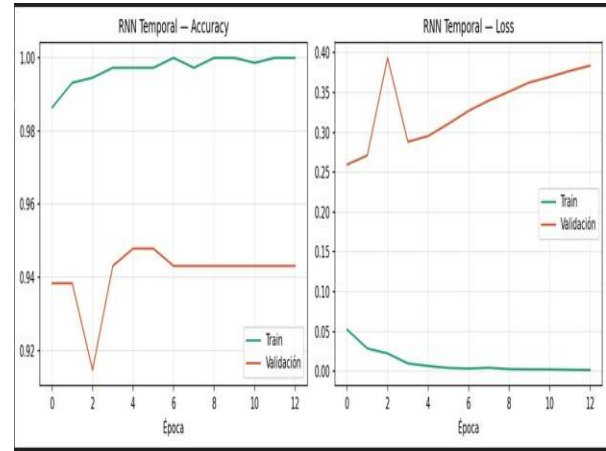


Fig. 3. Curvas de accuracy y loss del modelo BiLSTM (entrenamiento y validación). Convergencia en época 20.

C. Matrices de Confusión

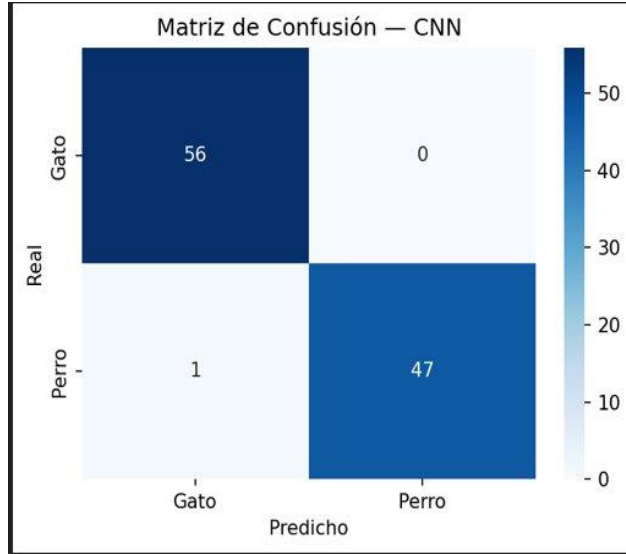


Fig. 4. Matriz de confusión del modelo CNN sobre el conjunto de test (heatmap azul).

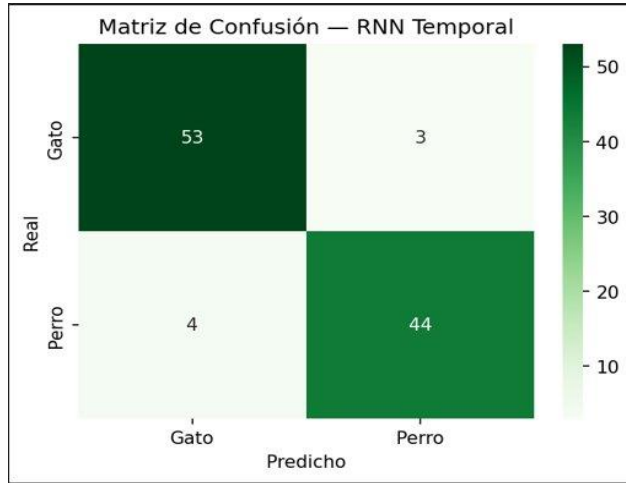


Fig. 5. Matriz de confusión del modelo BiLSTM sobre el conjunto de test (heatmap verde).

D. Comparativa Final CNN vs BiLSTM

Tabla VII. Comparativa completa CNN vs BiLSTM.

Métrica	CNN	BiLSTM (RNN)
Mejor val_accuracy	98.58%	94.79%
Test accuracy	97.12%	97.12%
Épocas entrenadas	13	20
Parámetros totales	~4.2 M	~99 K
Dominio de entrada	Frecuencia (Mel-Spec)	Frames Temporales
Tipo de representación	Imagen 128×130	Secuencia (130, 40)

E. Análisis de Errores de Clasificación

Cuando realizamos las matrices de confusión en la CNN, el único error fue clasificar 1 perro como gato, el modelo identificó correctamente los 56 gatos del conjunto de prueba utilizado, logrando una precisión perfecta en esa clase de audios, al momento de analizar la RNN temporal, se confundieron 3 gatos como perros y 4 perros como gatos (7 errores totales sobre 104 muestras), se tienen hipótesis de las posibles causas, en nuestro análisis como grupo, desde el punto de vista de la señal, podemos tener distintas causas, como pueden ser la variabilidad acústica dentro de cada clase, dado que los ladridos y maullidos cambian significativamente según el sujeto del que se tome y la intensidad, presencia de ruido de fondo en algunos clips que enmascara las características espectrales discriminantes; clips de muy corta duración en los hay introduce regiones silenciosas de la señal que distorsionan la amplitud media por frame, la RNN temporal opera sobre amplitud cruda sin compresión espectral, lo que la hace más sensible a variaciones de energía que no corresponden a diferencias de clase en la señal, lo que deja en evidencia la desventaja de trabajar la forma temporal pura de la señal tomada, por tanto, la CNN comete menos errores porque el mel-espectrograma comprime y resalta las frecuencias que ayudan a diferenciar entre maullidos y ladridos, mientras que la señal temporal cruda contiene mucha información irrelevante que dificulta la clasificación al entrenar al modelo.

La CNN es más sensible a variaciones en la textura visual del mel-espectrograma, mientras que la RNN BiLSTM es más sensible a la longitud y dinámica de la secuencia temporal de la señal, esto explica por qué ambos modelos pueden errar en ejemplos distintos aun cuando su accuracy global sea similar.

VII. INTERFAZ GRÁFICA DE USUARIO

Como componente adicional del proyecto, se implementó una interfaz gráfica interactiva en Google Colab mediante ipywidgets con ayuda de un modelo LLM para el frontend:

Explorador del test set: permite seleccionar mediante menús desplegables cualquier audio del conjunto de prueba por clase y nombre de archivo. Al presionar el botón 'Analizar Audio', se reproducen el audio seleccionado y se muestran simultáneamente la señal temporal, el mel-espectrograma (CNN) y los frames temporales (RNN), junto con las predicciones de ambos modelos y su confianza

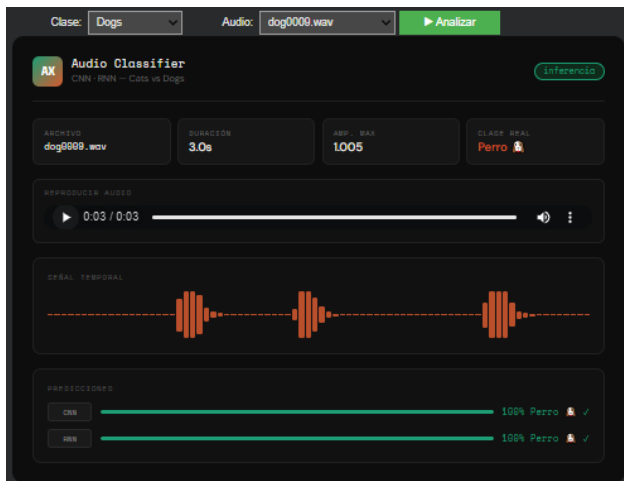


Fig. 6. Captura de la interfaz Audio Classifier Pro con resultado de predicción para un audio de prueba.

VIII. COMPARACIÓN Y DISCUSIÓN

La comparación entre la red CNN y RNN deja en evidencia diferencias fundamentales en la naturaleza de cada enfoque de modelo, permitiendo ver cual es mas, o menos preciso, dependiendo que tipo de señal se trabaje:

Representación del audio: el mel-espectrograma preserva simultáneamente información temporal y espectral de la señal en una misma imagen, lo que permite a la red CNN detectar patrones locales de tiempo y de frecuencia mediante filtros convolucionales, el framing temporal ofrece una representación directa de la parte espectral pero eliminan información de fase y de distribución espectral fina, lo cual es importante para la precisión.

Complejidad paramétrica: la CNN requiere mas o menos 4.2 M parámetros frente a los 99 K de la RNN BiLSTM. Esta diferencia se origina en la capa Flatten de la CNN, que genera un vector de alta dimensión, porque este lo que hace es aplanar los mapas convolucionales antes de la capa densa, así como dejar bien entendible la información, por lo cual el BiLSTM es significativamente más eficiente en términos de memoria, uso de recursos y tiempo de entrenamiento.

Velocidad de convergencia: la CNN convergió más lento (25 épocas vs 12) gracias a la naturaleza compacta y discriminativa de los patrones visuales en el mel-espectrograma, pero procesa mas información de la imagen para una mayor precisión, que son directamente aprovechables por los filtros convolucionales sin necesidad de modelar dependencias a largo plazo.

En tareas de clasificación de sonidos de animales, la representación espectral (mel-espectrograma) se caracteriza porque suele ser más informativa que la señal temporal cruda, dado que el timbre y la textura sonora de estas señales son las características más diferenciales entre especies, sin embargo, el BiLSTM captura la dinámica temporal de las “vocalizaciones”, lo que puede resultar ventajoso en clases donde el patrón de variación temporal sea la clave discriminante.

IX. CONCLUSIONES

Concluimos que en este proyecto demostró la viabilidad de ambos enfoques CNN usando mel-espectrogramas y RNN usando framing temporal, para el entrenamiento de la clasificación automática de sonidos de gatos y perros, los resultados obtenidos permiten establecer varias conclusiones:

El preprocesamiento basado en principios de procesamiento de señales (STFT, escala mel, framing temporal) es determinante para el desempeño y la precisión de los modelos, la elección para la representación de entrada condiciona directamente la arquitectura del modelo y la información que este puede aprovechar.

La CNN, al operar sobre imágenes de mel-espectrograma, captura patrones visuales tiempo-frecuencia de forma eficiente con early stopping en 25 épocas, pero esto cada vez que se ejecuta no tomara las mismas épocas, Su mayor número de parámetros (4.2 M) frente a los de la RNN refleja la riqueza de la representación espectral bidimensional, y la gran cantidad que de información con la que se puede trabajar.

La RNN, al procesar la señal temporal en frames de amplitud cruda, modela la dinámica temporal de las vocalizaciones de los animales. Su arquitectura bidireccional permite capturar información de la secuencia completa en ambas direcciones, siendo considerablemente más compacto 99 K parámetros y con la versatilidad de poder generalizar en datasets pequeños.

La interfaz gráfica implementada en Google Colab constituye una herramienta de validación práctica que permite verificar el comportamiento de ambos modelos.

X. REFERENCIAS

- [1] J. G. Proakis y D. G. Manolakis, Tratamiento Digital de Señales, 4.^a ed. Madrid, España: Pearson Educación, 2007.
 - [2] A. V. Oppenheim y R. W. Schaffer, Tratamiento de Señales en Tiempo Discreto, 3.^a ed. Madrid, España: Pearson, 2011.
 - [3] S. Hochreiter y J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, n.º 8, pp. 1735–1780, nov. 1997.
 - [4] B. McFee et al., "librosa: Audio and Music Signal Analysis in Python," en *Proc. 14th Python in Science Conf.*, Austin, TX, 2015, pp. 18–25. [En línea]. Disponible: <https://librosa.org>
 - [5] stealthtechnologies, "Cats vs Dogs Audio Classification," Kaggle, 2023. [En línea]. Disponible: <https://www.kaggle.com/datasets/stealthtechnologies/cats-vs-dogs-audio-classification>. [Accedido: 09-may-2026].
 - [6] TensorFlow, "tf.keras.layers.Bidirectional," TensorFlow Documentation. [En línea]. Disponible: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Bidirectional. [Accedido: 09-may-2026].
 - [7] F. Chollet, *Deep Learning with Python*, 2.^a ed. Shelter Island, NY: Manning Publications, 2021.
-